

Module 4: Arrays & Objects in JavaScript

Arrays and Objects are the most important data structures in JavaScript. They are used extensively in web development, APIs, React, Node.js, and modern JavaScript applications.

1. Arrays

An **Array** is used to store multiple values in a single variable.

Example

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
console.log(fruits);
```

Output

```
["Apple", "Mango", "Banana"]
```

Array Methods

1. push()

Adds element at the end.

```
let fruits = ["Apple", "Mango"];
```

```
fruits.push("Banana");
```

```
console.log(fruits);
```

Output

```
["Apple", "Mango", "Banana"]
```

2. pop()

Removes last element.

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
fruits.pop();
```

```
console.log(fruits);
```

Output

```
["Apple", "Mango"]
```

3. shift()

Removes first element.

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
fruits.shift();
```

```
console.log(fruits);
```

Output

```
["Mango", "Banana"]
```

4. unshift()

Adds element at beginning.

```
let fruits = ["Mango", "Banana"];
```

```
fruits.unshift("Apple");
```

```
console.log(fruits);
```

Output

```
["Apple", "Mango", "Banana"]
```

5. splice()

Add, remove, or replace elements.

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
fruits.splice(1, 1, "Orange");
```

```
console.log(fruits);
```

Output

```
["Apple", "Orange", "Banana"]
```

6. slice()

Creates a new array.

```
let fruits = ["Apple", "Mango", "Banana", "Orange"];
```

```
let result = fruits.slice(1, 3);
```

```
console.log(result);
```

Output

```
["Mango", "Banana"]
```

7. includes()

Checks if value exists.

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
console.log(fruits.includes("Mango"));
```

Output

```
true
```

8. indexOf()

Returns position.

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
console.log(fruits.indexOf("Banana"));
```

Output

```
2
```

9. map()

Creates new array after transformation.

```
let numbers = [1, 2, 3, 4];
```

```
let square = numbers.map(num => num * num);
```

```
console.log(square);
```

Output

```
[1, 4, 9, 16]
```

10. filter()

Filters matching values.

```
let numbers = [10, 20, 30, 40, 50];
```

```
let result = numbers.filter(num => num > 25);
```

```
console.log(result);
```

Output

```
[30, 40, 50]
```

11. reduce()

Reduces array to single value.

```
let numbers = [10, 20, 30];
```

```
let total = numbers.reduce((sum, num) => sum +  
num, 0);
```

```
console.log(total);
```

Output

60

Practical Example: Shopping Cart

```
let prices = [500, 1000, 1500];
```

```
let total = prices.reduce(  
  (sum, price) => sum + price,  
  0  
);
```

```
console.log("Total =", total);
```

Output

Total = 3000

2. Objects

An Object stores data as key-value pairs.

Example

```
let student = {  
  name: "Vicky",  
  age: 20,  
  course: "BCA"  
};
```

```
console.log(student);
```

Output

```
{name:"Vicky", age:20, course:"BCA"}
```

Object Methods

1. Object.keys()

Returns all keys.

```
let student = {  
  name: "Vicky",  
  age: 20  
};
```

```
console.log(Object.keys(student));
```

Output

```
["name", "age"]
```

2. Object.values()

Returns values.

```
console.log(Object.values(student));
```

Output

```
["Vicky", 20]
```

3. Object.entries()

Returns key-value pairs.

```
console.log(Object.entries(student));
```

Output

```
[["name","Vicky"],["age",20]]
```

4. Object.assign()

Copies objects.

```
let obj1 = {a:1};
```

```
let obj2 = {b:2};
```

```
let result = Object.assign({}, obj1, obj2);
```

```
console.log(result);
```

Output

```
{a:1, b:2}
```

Practical Example: Employee Object

```
let employee = {  
  name: "Amit",
```

```
salary: 50000,  
department: "IT"  
};
```

```
Object.entries(employee).forEach(  
  ([key, value]) => {  
    console.log(key + ":", value);  
  }  
);
```

3. Destructuring

Destructuring extracts values from arrays or objects.

Array Destructuring

```
let colors = ["Red", "Green", "Blue"];
```

```
let [first, second, third] = colors;
```

```
console.log(first);
```

```
console.log(second);
```

Output

Red
Green

Object Destructuring

```
let student = {  
  name: "Vicky",  
  age: 20  
};
```

```
let {name, age} = student;
```

```
console.log(name);  
console.log(age);
```

Output

Vicky
20

Practical Example

```
let employee = {  
  name: "Rahul",  
  salary: 60000
```

```
};  
  
let {name, salary} = employee;  
  
console.log(  
  `${name} earns ₹${salary}`  
);
```

Output

Rahul earns ₹60000

4. Spread Operator (...)

The Spread Operator expands arrays or objects.

Array Spread

```
let arr1 = [1, 2, 3];  
  
let arr2 = [...arr1, 4, 5];  
  
console.log(arr2);
```

Output

[1,2,3,4,5]

Merge Arrays

```
let boys = ["Amit", "Rahul"];  
let girls = ["Pooja", "Neha"];  
  
let students = [...boys, ...girls];  
  
console.log(students);
```

Output

```
["Amit","Rahul","Pooja","Neha"]
```

Object Spread

```
let student = {  
  name: "Vicky"  
};  
  
let details = {  
  ...student,  
  age: 20,  
  city: "Jaipur"
```

```
};
```

```
console.log(details);
```

Output

```
{name:"Vicky", age:20, city:"Jaipur"}
```

Real-Life Example: E-Commerce Cart

```
let cart1 = ["Laptop", "Mouse"];
```

```
let cart2 = ["Keyboard", "Headphone"];
```

```
let finalCart = [...cart1, ...cart2];
```

```
console.log(finalCart);
```

Output

```
["Laptop","Mouse","Keyboard","Headphone"]
```

Comprehensive Example

```
let students = [
```

```
{
```

```
  name: "Amit",
```

```
marks: 85
},
{
  name: "Rahul",
  marks: 70
},
{
  name: "Pooja",
  marks: 95
}
];

let topper = students
  .filter(student => student.marks > 80)
  .map(student => student.name);

console.log(topper);
```

Output

["Amit", "Pooja"]

Summary Table

Topic	Purpose
-------	---------

Topic	Purpose
push()	Add at end
pop()	Remove from end
shift()	Remove from start
unshift()	Add at start
splice()	Add/Remove/Replace
slice()	Copy part of array
map()	Transform data
filter()	Filter data
reduce()	Single value result
Object.keys()	Get keys
Object.values()	Get values
Object.entries()	Get key-value pairs
Object.assign()	Merge/Copy objects
Destructuring	Extract values easily
Spread Operator	Expand/merge arrays & objects

